

PRESENTATION PROBLEMS

Instructor: Orhan Özgüner

© Özgüner @CWRU, 2025

Presentation 1

You are working with an independent comics publisher to help digitize and organize their archives. Comics are usually given issue numbers that are positive integers, although as a promotional gimmick some have 0 or a negative number. The archive is not sorted, and may contain duplicate issues. More importantly, some issues are missing from it. Your task is to find the *smallest positive integer* issue number that is *not* present in the archive (which is treated as an array of integers A).

That is, formally, you want to develop an efficient algorithm to find the *smallest* integer s such that $s > 0$ and $A[i] \neq s \forall 1 \leq i \leq |A|$.

Example:

Input:	$A = [10, 5, 12, 1, 11]$	$A = [-3, 0, -1, -5]$	$A = [1, 1, 2, 3]$	$A = []$
Output:	2	1	4	1

Presentation 1 Solution

Pseudocode

Algorithm 1

Require: List A of n integers.

Ensure: s is the smallest integer not in A .

```
1: function FINDMISSINGISSUE( $A, n$ )
2:    $i \leftarrow 1$ 
3:   while  $i < n$  do                                 $\triangleright$  Sort elements into their corresponding positions
4:     if  $1 \leq A[i] \leq n$  and  $A[A[i]] \neq A[i]$  then
5:       Swap ( $A[i] \longleftrightarrow A[A[i]]$ )           $\triangleright$  Place value  $A[i]$  at index  $A[A[i]]$ 
6:     else
7:        $i \leftarrow i + 1$ 
8:     end if
9:   end while
10:  for  $i \leftarrow 0$  to  $n$  do                           $\triangleright$  Scan for the first index where value is incorrect
11:    if  $A[i] \neq i$  then
12:      return  $i$                                         $\triangleright$  Found the smallest missing positive integer
13:    end if
14:  end for
15:  return  $n + 1$   $\triangleright$  All values from 1 to  $n$  are present. Return next integer beyond array length
16: end function
```

Intuition

This algorithm exploits the fact that the maximum possible value of s in an array of size n , is surprisingly small- $n + 1$. This is because, to make the value of s go up, every positive integer from 1 to $s - 1$ needs to be present in the array, and the array only has room for n integers. This also means that values in the array < 1 or $> n + 1$, can have no effect on s .

To exploit this, the algorithm performs a simplified counting sort that ignores values < 1 or $> n + 1$. It attempts to place each valid value x (where $1 \leq x \leq n$) at index x . This transforms the array into a form where, ideally, each position $A[i]$ contains the value i if and only if i was present in the array.

After this rearrangement, the algorithm performs a linear scan: the first index i where $A[i] \neq i + 1$ indicates that $i + 1$ is missing from the array. After this rearrangement, the algorithm performs a linear scan: the first index i where $A[i] \neq i + 1$ indicates that $i + 1$ is missing from the array.

Proof of Correctness

First Loop Invariant (FL)

- **Invariant:** The loop invariant consists of two clauses.

1. $A[x] \neq x$ for the smallest x *not* present in subarray $A[1...i]$, and
2. For all values y that *are* present, $A[y] = y$.

- **Initialization:**

1. At the start of the loop, the region $A[1\dots i]$ is empty, so Clauses 1 and 2 are both vacuously true.

- **Maintenance:** Suppose that the loop invariant was true for iteration $i - 1$. Then after another iteration, there are 5 cases:

1. If $A[i] = i$, then do nothing. Increment i and move on to the next iteration. This occurs if element i was in the subarray $A[1\dots i - 1]$.
2. If $A[i] \leq 0$, then do nothing. Increment i and move on to the next iteration. This implies element i was not in subarray $A[1\dots i - 1]$, so i is a missing element of $A[1\dots i]$.
3. For convenience, use the shorthand $A[i] = k$. If $0 < k < i$, then we will swap the elements at $A[i]$ and $A[k]$. Then $A[k]$ will be correct, and $A[i]$ will be incorrect. But we know i is a missing element (as we would otherwise go to case 1).
4. For convenience, use the shorthand $A[i] = j$. If $i < j < n$, swap $A[i]$ and $A[j]$. So, when we get to index j , $A[j] = j$.
5. If $A[i] > n$, do nothing. Equivalent to Case 2.

- **Termination:** The for loop increments i by 1 in each iteration if $A[i] = i$, and no operation in the loop decrements it. If i is not incremented, the number of places where $A[x] = x$ increases by at least 1. Since A is of finite size, it must eventually set $A[i] = i$ for each i . The loop is thus guaranteed to terminate. When it does, $i = n + 1$, so the loop invariant becomes “ $A[x] \neq x$ for the smallest x *not* present in subarray $A[1\dots i]$ ”, or simply $A[x] \neq x$ for the smallest x *not* present in the entire array A ”.

Second Loop Invariant (SL)

- **Invariant:** Every value x such that $1 \leq x < i$ is present in A .
- **Initialization:** At initialization, there are no values x such that $1 \leq x < 1$, so we do not need to check if anything is in A .
- **Maintenance:** Assume every value x such that $1 \leq x < i - 1$ is present in A . From L1, either $A[x] = x$ or x is not in A . Loop 2 terminates if $A[i] \neq i$, so it terminates if i is not present in A . If it does *not* terminate (i.e. we make it to the end of the iteration), we can say that every value x such that $1 \leq x < i - 1$ is present in A , and that i is present in A , so every value x such that $1 \leq x < i$ is present in A .
- **Termination:** The loop terminates for one of two reasons: i is not present in A , or $i = n + 1$. If i is not present in A , then every value x such that $1 \leq x < i - 1$ must still be present in A , so i is the smallest value not present. If $i = n + 1$, then every value x such that $1 \leq x < n + 1$ must be present.

Runtime Analysis

The `FindMissingIssue` algorithm consists of two main steps: an in-place rearrangement of the array and a linear scan to identify the smallest missing positive integer.

Step 1: Rearrangement Loop

- The outer loop runs from $i = 0$ to $n - 1$, where n is the length of the array.
- For each index i , the algorithm enters a **while** loop that swaps $A[i]$ with $A[A[i] - 1]$ if:
 - $1 \leq A[i] \leq n$ (i.e., the value is within bounds), and
 - $A[A[i] - 1] \neq A[i]$ (i.e., the value is not already in its correct position).
- Each swap places a value closer to its correct index. Since each value is moved at most once to its final position, the total number of swaps is bounded by n .
- Each swap and comparison is a constant-time operation, i.e., $\mathcal{O}(1)$.

Therefore, the total time complexity of the rearrangement phase is $\mathcal{O}(n)$.

Step 2: Scan Loop

- After rearrangement, the algorithm performs a single pass over the array from $i = 0$ to $n - 1$.
- At each index, it checks whether $A[i] = i + 1$. If not, it returns $i + 1$ as the smallest missing positive integer.
- Each comparison is a constant-time operation, i.e., $\mathcal{O}(1)$.

Thus, the scan phase also runs in $\mathcal{O}(n)$ time.

Overall Complexity

- **Time Complexity:** $\mathcal{O}(n)$
- **Space Complexity:** $\mathcal{O}(1)$ (in-place, no extra memory used)

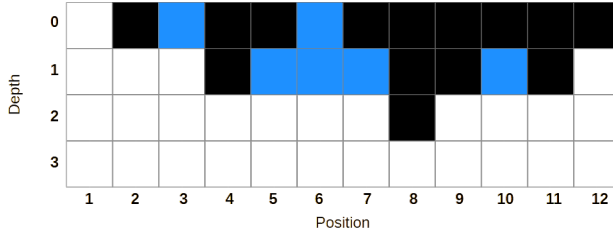
Presentation 2

A geothermal energy company is extracting steam from an area with a layer of solid rock on top of a layer of porous rock. The steam forms deep underground and bubbles upward until it hits solid rock. If there is not either solid rock or steam directly beside a bubble, it can move to the side and try to continue upward, but it cannot go downward. So, steam accumulates in *minima* of the porous rock layer (see the example picture). Steam that moves outside of the area completely, is lost.

You have a geology report showing the depth of the porous layer at regular intervals (the array A included under the image). The power company wants to know *how much steam, in total* is available in the region. Develop an efficient algorithm to solve this problem.

Example:

Input: $A = [0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]$



Output: Total Amount: 6

Presentation 2 Solution

Pseudocode

Algorithm 2

Require: List A of n solid rock depth measurements.

Ensure: s is the total amount of steam that can be contained in the formation described by A .

```
1: function CALCULATESTEAM( $A, n$ )
2:    $left \leftarrow 1$                                 ▷ Start pointer from the left end
3:    $right \leftarrow n$                                ▷ Start pointer from the right end
4:    $left\_max \leftarrow A[1]$ 
5:    $right\_max \leftarrow A[n]$                         ▷ Track min and max depth seen from the left and right
6:    $total \leftarrow 0$                                 ▷ Total steam trapped
7:   while  $left + 1 \neq right$  do
8:     if  $A[left] < A[right]$  then                    ▷ Process the side with lower boundary
9:       if  $A[left] \geq left\_max$  then
10:         $left\_max \leftarrow A[left]$                 ▷ Update left max if current is higher
11:       else
12:         $total \leftarrow total + (left\_max - A[left])$  ▷ Steam trapped at current left position
13:       end if
14:        $left \leftarrow left + 1$                       ▷ Move left pointer
15:     else
16:       if  $A[right] \geq right\_max$  then
17:         $right\_max \leftarrow A[right]$               ▷ Update right max if current is higher
18:       else
19:         $total \leftarrow total + (right\_max - A[right])$  ▷ Steam trapped at current right position
20:       end if
21:        $right \leftarrow right - 1$                   ▷ Move right pointer
22:     end if
23:   end while
24:   return  $total$                                     ▷ Return total steam accumulated
25: end function
```

Intuition

This algorithm models how steam accumulates beneath a layer of solid rock by analyzing the terrain formed by the porous rock layer. The input array represents the depth of porous rock at regular intervals, and steam naturally rises until blocked by solid rock. If it encounters a barrier, it can move sideways—but never downward—so it tends to collect in valleys between higher rock formations.

To calculate how much steam is trapped, the algorithm uses two pointers—one starting from the left and one from the right—and tracks the tallest rock seen so far from each direction. At each step, it processes the side with the lower boundary, since that side determines the maximum steam that can be trapped at that position. If the current depth is less than the maximum seen so far, the difference represents trapped steam. This continues until the two pointers meet, ensuring every position is evaluated exactly once.

Proof of Correctness

We will refer to the "steam function" of a cell $A[i]$, the amount of steam trapped in that cell, as $S(i)$. The total amount of steam in a region $A[x...y]$ is similarly written $S(x...y)$.

- **Invariant:** The invariant will have three clauses:

1. $left_max$ is the deepest rock formation (maximum value) in the region $[1...l]$.
2. $right_max$ is the deepest rock formation (maximum value) in the region $[r...n]$.
3. $total = S(1...left) + S(right...n)$

- **Initialization:**

1. At initialization, $A[1...left]$ has one element, $A[1]$. By Line 4, $left_max = A[1]$. $\square$
2. At initialization, $A[right...n]$ has one element, $A[n]$. By Line 5, $right_max = A[n]$. $\square$
3. Neither $A[1]$ nor $A[n]$ can trap any steam, because they are on the edges of the formation (and steam can always escape to the left or right side, respectively), so $(1...left) + S(right...n) = S(1) + S(n) = 0$. $total = 0$ by Line 6. $\square$

- **Maintenance:**

If $A[left] < A[right]...$

1. $left_max$ becomes the larger of two values: the previous $left_max$, or $A[left]$. Since $left_max$ was previously the maximum value in the region $[1...left - 1]$, it is now the maximum value in the region $[1...left]$. $\square$
2. $right$ is not incremented, so $right_max$ is still the maximum value in the region $[right...n]$. $\square$
3. We begin with $total = S(1...left - 1) + S(right...n)$. Since $A[left] < A[right]$, steam can get over the left wall more easily than it can get over the right. $S(left) = left_max - A[left]$, if $A[left] < left_max$, or $S(left) = 0$ otherwise. If $A[left] < left_max$, $total = total + (left_max - A[left]) = S(1...left - 1) + S(left) + S(right...n) = S(1...left) +$

$S(right...n)$. Otherwise, if $A[left] \geq left_max$, the if statement on Line 9 is triggered, the else statement on line 11 is *not* triggered, and $total$ is not updated, so $total = total + 0 = S(1...left - 1) + S(left) + S(right...n) = S(1...left) + S(right...n)$. $\square$

The proof for $A[left] \leq A[right]$ follows symmetrically, with the justifications for Clauses 1 and 2 switched.

- **Termination:**

$left$ and $right$ are initialized to integer values with $right > left$, and either $left$ increases by 1, **or** $right$ decreases by 1, every iteration of the while loop. The while loop must therefore eventually reach a position where $left + 1 = right$, and terminate. Clause 3 of the loop invariant states “ $total = S(1...left) + S(right...n)$ ”, so when $left + 1 = right$, this becomes equivalent to “ $total = S(1...left) + S(left + 1...n)$ ”, or “ $total$ is the amount of steam in $A[1...n]$ ”.

$\square$

Runtime Analysis

Two pointers, $left$ and $right$, are initialized at the beginning and end of the array, respectively. At each iteration, the algorithm compares $A[left]$ and $A[right]$ to decide which side to process. The pointer on the side with the lower value is moved inward. This ensures that steam is only counted when it is bounded by a higher wall on the opposite side.

Since each pointer moves inward by one position per iteration and they never move backward, the total number of iterations is at most n , where n is the length of the array.

Constant-Time Operations Per Iteration

Each iteration performs a fixed number of operations:

- **Comparison:** $A[left] < A[right]$ to determine which side to process.
- **Max Update:** Update $left_max$ or $right_max$ if the current value is greater than or equal to the tracked maximum.
- **Steam Calculation:** If the current value is less than the tracked maximum, compute trapped steam as $max - A[i]$.
- **Pointer Update:** Increment $left$ or decrement $right$.

All of these operations are constant-time, i.e., $\mathcal{O}(1)$.

Total Work Done

Operation Type	Per Iteration	Total Across Array
Comparisons	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Max Updates	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Steam Calculations	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Pointer Movement	$\mathcal{O}(1)$	$\mathcal{O}(n)$

Final Complexity

- Time Complexity: $\mathcal{O}(n)$
- Space Complexity: $\mathcal{O}(1)$